

# CI/CD

Version: 1.0

---

## Purpose

This document defines the Continuous Integration and Continuous Delivery (CI/CD) strategy of Trading Platform Pro.

The objective is to automate quality assurance, reduce integration risks and ensure a consistent software delivery process.

---

## Objectives

The CI/CD pipeline shall provide:

- automated quality checks
- repeatable builds
- automated testing
- consistent formatting
- reliable deployments
- fast developer feedback

---

## CI Principles

Every change should be automatically verified before integration.

The pipeline should be:

- deterministic
- reproducible
- fast
- reliable
- transparent

---

## Continuous Integration

Every push should automatically execute:

1. Checkout Repository
2. Setup Python
3. Install Dependencies
4. Ruff Check
5. Ruff Format Verification
6. Unit Tests
7. Integration Tests (when available)

Pipeline failures must block integration until resolved.

---

## Continuous Delivery

Future delivery stages may include:

- Build Artifacts
- Versioning
- Release Packaging
- Deployment Validation
- Release Notes

Deployment automation should remain independent from application logic.

---

## Branch Strategy

Primary branch:

---

main

---

Optional feature branches:

---

feature/

---

Bug fixes:

---

bugfix/

---

---

## Code Quality Gates

Before integration the following must succeed:

- Ruff
- Formatting
- Unit Tests
- Integration Tests
- Documentation validation (where applicable)

---

## Dependencies

Development dependencies are maintained separately from production dependencies.

Examples:

- requirements.txt
- requirements-dev.txt

---

## GitHub Actions

CI is implemented using GitHub Actions.

Typical workflow:

---

Checkout

↓

Setup Python

↓

Install Dependencies

↓

Run Ruff

↓

Run Tests

↓

Publish Results

---

---

## Local Verification

Developers should execute before pushing:

```
```bash
```

```
ruff check .
```

```
ruff format .
```

```
pytest tests/unit -q
```

```
```
```

This reduces unnecessary CI failures.

---

## Future Improvements

Potential enhancements include:

- coverage reporting
- security scanning
- dependency auditing
- automated releases
- container builds
- deployment automation

---

## Release Readiness

Before a release verify:

- all quality gates passed
- documentation updated
- version numbers updated
- changelog completed
- release notes prepared

Only release software that satisfies all quality criteria.

---

## CI/CD Review Checklist

Before enabling pipeline changes verify:

- reproducible builds
- deterministic execution
- successful test execution
- reliable quality gates
- documented workflow
- rollback strategy available

---

## Related Documents

- Git\_Workflow.md
- Testing\_Strategy.md
- Development\_Guidelines.md
- Runtime.md
- AGENTS.md